САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэкэнд разработка

Отчет

Лабораторная работа №4

Выполнил:

Михайлов Юрий

Группа К3441

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- провести предварительную настройку удалённого сервера (установка необходимых зависимостей);
- провести развёртывание приложения на удалённом сервере;
- настроить nginx в режиме обратного прокси для обеспечения доступа к вашему приложению извне.

Ход работы

1. Создание nginx конфига для проксирования запросов

На удаленном сервере, на котором производилось развертывание приложения, был настроен доступ по ssh и базовый nginx, который проксировал запросы с 81 порта на 127.0.0.1:8000.

В docker-compose.yml был добавлен сервис gateway на образе nginx:alpine. Он публикует порт 8000:80 (снаружи 8000, внутри контейнера 80) и монтирует наш конфиг nginx/nginx.conf. Все остальные сервисы были лишены блоков ports, чтобы наружу ничего не торчало. Сервисы общаются между собой по внутренней Docker-сети rental-network по DNS-именам (auth-service, property-service и т.д.).

В nginx/nginx.conf описаны upstream-блоки для каждого сервиса и location-блоки, которые распределяют запросы по путям:

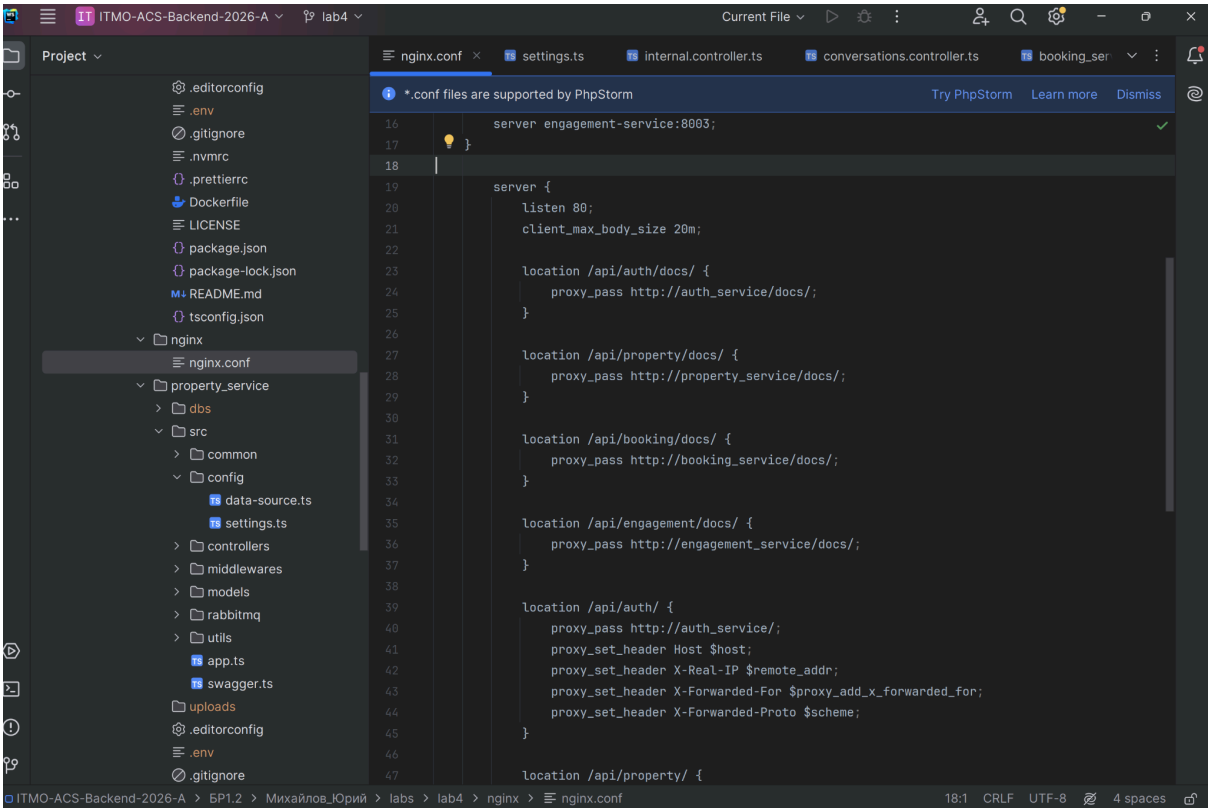
- /api/auth/... - auth-service:8000
- /api/property/... - property-service:8001
- /api/booking/... - booking-service:8002
- /api/engagement/... - engagement-service:8003

Для Swagger UI каждого сервиса сделаны отдельные location (/api/<service>/docs/), которые проксируют на /docs/ внутри сервиса, потому что Swagger у всех сервисов живёт по одному и тому же пути /docs/, и без отдельного префикса они бы конфликтовали.

Отдельно пришлось разобраться со слешем на конце proxy_pass: если в location /api/property/ написать proxy_pass http://property-service; (без слеша), Nginx передаёт путь целиком, и в сервис уходит /api/property/api/properties, чего сервис не знает. Если написать proxy_pass

http://property-service/; (со слешем), Nginx отрезает префикс /api/property/, и в сервис уходит /api/properties — то, что он ждёт. Поэтому для бизнес-роутов используется вариант со слешем.

Также был настроен healthcheck для gateway (через wget на /health), чтобы Docker Compose мог отслеживать его состояние.



2. Настройка и проверка относительных путей

Внутри сервисов пути уже имеют префикс /api/... (например, GET /api/properties в property-service, POST /api/booking-requests в booking-service). Поэтому снаружи полный URL получается с двойным префиксом: /api/property/api/properties, /api/booking/api/booking-requests/my и т.д. Первый /api/<service>/ — это то, что добавляет наш gateway, второй /api/... — путь внутри сервиса.

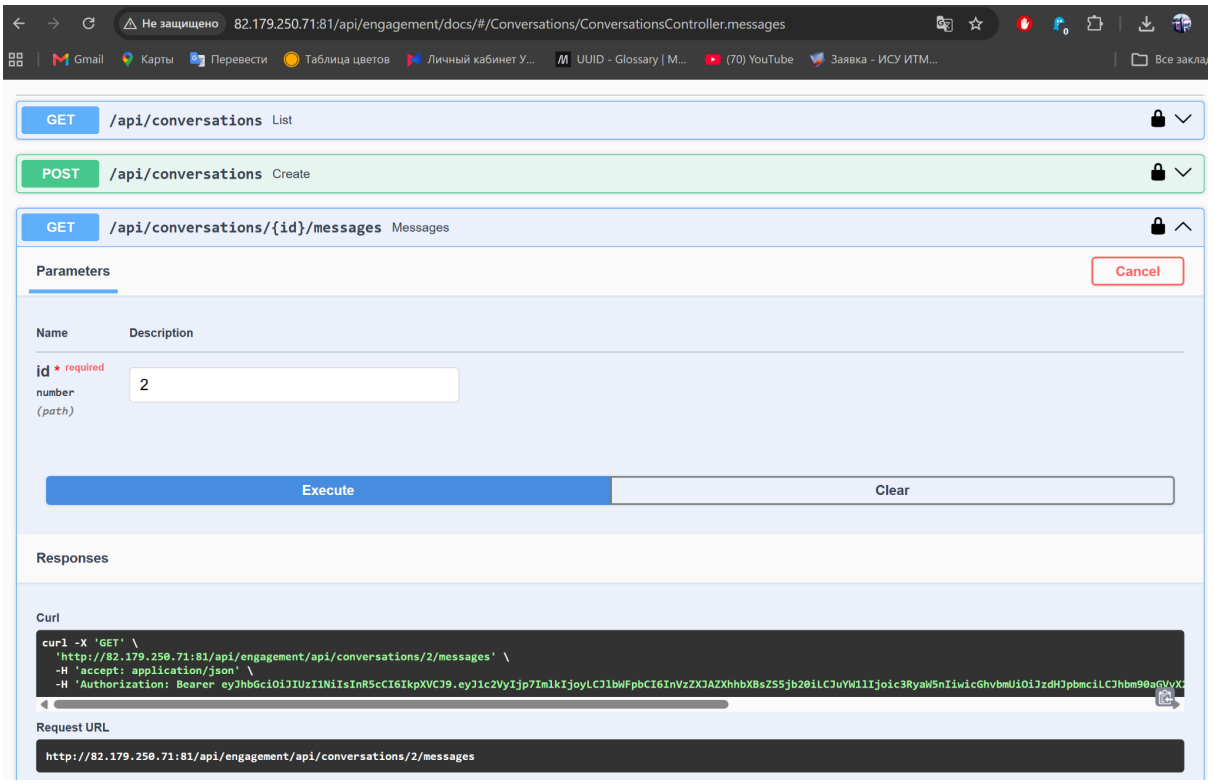
Чтобы Swagger UI не путал пользователя и при нажатии «Try it out» отправлял запросы на правильный внешний URL, в настройки Swagger каждого сервиса было добавлено поле servers:

- auth → servers: [{ url: '/api/auth' }]
- property → servers: [{ url: '/api/property' }]

- booking → servers: [{ url: '/api/booking' }]
- engagement → servers: [{ url: '/api/engagement' }]

Это заставляет Swagger добавлять нужный префикс к запросам. В коде это делается через routingControllersToSpec(storage, options, { ..., servers: [{ url: '/api/booking' }] }).

Также были проверены относительные пути для внутренних вызовов между сервисами (например, booking-service ходит в property-service по http://property-service:8001/api/internal/properties/{id}). Эти вызовы идут по внутренней Docker-сети и не затрагивают Nginx, поэтому после добавления gateway они продолжили работать без изменений.



3. Проверка результатов

В ходе ручной проверки нескольких сценариев (создание пользователя, создание объявления, запрос на бронирование, подтверждение бронирования, общение в диалоге, получение списка всех объектов/броней) было установлено, что весь функционал работает корректно и успешно перенесся на удаленный сервер.

Вывод

В ходе работы приложение было успешно развёрнуто на удалённом сервере: настроены Docker Compose, внутренний Nginx-гейтвей и обратный прокси, благодаря чему снаружи доступен только один порт 8000, а все четыре микросервиса — через свои префиксы /api/<service>/. Проверка показала, что Swagger UI каждого сервиса и бизнес-эндпоинты доступны извне, базы данных и RabbitMQ изолированы во внутренней Docker-сети, а межсервисные вызовы продолжают работать без изменений.